



An academic recommender system on large citation data based on clustering, graph modeling and deep learning

Vaios Stergiopoulos¹ · Michael Vassilakopoulos¹ · Eleni Tousidou¹ · Antonio Corral²

Received: 22 August 2023 / Revised: 10 January 2024 / Accepted: 6 March 2024 /
Published online: 18 April 2024
© The Author(s) 2024

Abstract

Recommendation (recommender) systems (RS) have played a significant role in both research and industry in recent years. In the area of academia, there is a need to help researchers discover the most appropriate and relevant scientific information through recommendations. Nevertheless, we argue that there is a major gap between academic state-of-the-art RS and real-world problems. In this paper, we present a novel multi-staged RS based on clustering, graph modeling and deep learning that manages to run on a full dataset (scientific digital library) in the magnitude of millions users and items (papers). We run several tests (experiments/evaluation) as a means to find the best approach regarding the tuning of our system; so, we present and compare three versions of our RS regarding recall and NDCG metrics. The results show that a multi-staged RS that utilizes a variety of techniques and algorithms is able to face real-world problems and large academic datasets. In this way, we suggest a way to close or minimize the gap between research and industry value RS.

Keywords Recommender systems · K-means clustering · Graph based · Neural networks · Deep learning · AMiner citation network

1 Introduction

Recommender systems (RS) are among the most utilized and successful applications of artificial intelligence (AI) and machine learning (ML) technology in practice. Nowadays,

✉ Antonio Corral
acorral@ual.es

Vaios Stergiopoulos
vstergiop@uth.gr

Michael Vassilakopoulos
mvasilako@uth.gr

Eleni Tousidou
etousido@uth.gr

¹ Data Structuring and Engineering Lab, Department of Electrical and Computer Engineering, University of Thessaly, Volos, Greece

² Department of Informatics, University of Almeria, Almeria, Spain

such systems are used daily online for a huge variety of applications, for example on e-commerce sites, on media streaming platforms, in social networks or in academic digital libraries. RS help us in academic digital libraries by suggesting items (scientific publications) that are assumed to be of interest to us and which we are correspondingly likely to read, study or purchase.

Given the insights from the literature, it is undisputed that *Academic RS (or publications RS)* can have positive effects in a variety of ways, for both the researchers/authors of a paper and for the researcher who is searching for scientific information relevant to his/her academic interests.

Generally speaking, the common modeling methods used in RS in previous decades featured: Content-based Filtering (CBF), Collaborative Filtering (CF) or Knowledge-based Filtering (KBF). In this regard, various hybrid techniques have been created in an effort to gain advantages while disregarding the disadvantages of each technique.

In recent years, an alternative approach has emerged, the representation learning-based methods that try to encode both users and items as continuous vectors (i.e., vector embeddings) in order to make them directly comparable. Representation learning-based models include Matrix Factorization (MF) models, Artificial Neural Networks (ANN), Deep Learning (DL) models and Graph Learning-based models. Due to their capacity to accurately capture the nonlinear and non-trivial user-item connections, Deep Learning models have recently been a prominent paradigm for RS in both scholarly research and applications in industry. Additionally, graph learning-based methods have been used, which consider the information in RS from the perspective of graphs, due to the underlying graph structure that is usually implied by the data used in an RS.

Recently, Hyper-Parameters (HP) tuning of Deep Neural Networks (DNN) in RS has risen as a principal topic. The achieved DNN's performance is extremely sensitive to the HP selection and setting. The activation function is a valuable part of an ANN because it turns a linear classifier into a nonlinear one, which has proved to be the key factor to improve the recommendation performance. The right initialization method of the ANN's weights is very critical to the resulting convergence. Furthermore, the number of training epochs has a huge impact in the training quality of the ANN. The selection of the training epochs number is highly relevant to two problems, under-fitting and over-fitting of the model; very low learning and inadequate fitting of the training data versus extremely high learning and fitting on the training data, respectively. In both cases, the DL model cannot make successful recommendations for the test data.

While Deep Learning models have been extensively used in RS, it is common knowledge that they are extremely demanding applications in computational power and memory, when dealing with large datasets consisting of many users and/or items. Usually state-of-the-art ML RS in the literature are limited in some thousands of users and items. Therefore, it is a fact that there is a major gap between the size of the datasets used in state-of-the-art published RS and the real-world datasets available today. Consequently, one of the main motivations of this paper is to try to cover the gap between the state-of-the-art RS approaches and real-world problems or (large) real-world datasets. Therefore, we felt the impulse to develop an algorithm and create a system that is able to utilize a full real-world dataset (million scale papers-users) in the process of making recommendations to users.

In this work, we aim to incorporate a variety of techniques and algorithms in a multi-stage, academic RS which uses a full dataset available in real-world (millions of users-papers scale) and achieves high evaluation scores in recommendations. For this purpose, we use techniques and algorithms like Clustering, Graph-modeling and Deep Learning (DL). Moreover, our system is hybrid with respect to recommendation techniques, as CBF and CF are incorporated.

Also, in this work, we embody an HP-tuned DL RS [1], the CATA++, an abbreviation for the full name: Collaborative Dual Attentive Auto-encoder method for recommending publications [2]. In order to HP tune CATA++, lots of activation functions, weight initializers and training epoch numbers were selected for the testing phase (evaluation) [1].

The remainder of this paper is organized in the following order. First, all related work is discussed in Sect. 2. Secondly, the proposed system is presented in Sect. 3. Next, our experimental results are demonstrated in Sect. 4. In Sect. 5, we discuss about the experimental results of our system. Finally, in Sect. 6 we conclude this work and discuss related future work directions.

2 Related work

Since the first research articles in the mid-1990s and the growth of application fields like e-commerce, online shopping, digital scientific journals, etc., RS have been an increasingly prominent area of study. RS consist of algorithms and software that try to offer consumers tailored recommendations that assist them deal with information overload and support decision-making. The RS produced a set of recommendations, which might take many different forms depending on the situation. Some examples of these goods include films, music, merchandise and scientific papers. These days, the literature has documented a number of RS-building techniques; here, we highlight those that are most pertinent to our case study and most illustrative of them.

Zhang et al. [3] systematically go over the fundamental approaches and methods used in RS, as well as how AI can help RS be applied and developed more successfully. They examine innovative theoretical and practical contributions, but they also point out current research problems and suggest new areas for future research. They extensively examine numerous RS-related AI difficulties and examine how these systems have been improved utilizing AI techniques like fuzzy techniques, transfer learning, genetic algorithms, evolutionary algorithms, neural networks and deep learning. In an effort to get around the drawbacks of the single-technique methods, hybrid approaches have been introduced that combine two or more techniques trying to overcome the limitations of each one. Today, most of the state-of-the-art RS are hybrid, combining a number of technologies or techniques.

The authors in [3] classified recommendation systems into the following categories:

- Content-based Filtering (CB or CBF) approaches
- Collaborative Filtering (CF) approaches
- Knowledge-based Filtering (KB or KBF) approaches
- Machine Learning (ML)/ Deep Learning (DL) approaches
- Graph-based approaches [4, 5]
- Hybrid approaches

Moreover, Kreutz and Schenkel [6] accomplish a literature study and review of recent RS publications; they present the used algorithms, datasets, evaluations and open challenges encountered in papers released between January 2019 and October 2021. They aim to provide a comprehensive, detailed and complete guide of the current state-of-the-art RS.

Beheshti et al. [7] talk about the shortcomings of current state-of-the-art approaches to RS and the need for their new strategy, which consists of a general framework and vision for a new kind of data-driven, knowledge-driven and cognition-driven RS called cognitive RS (cognition is the comprehension of the character, personality, behavior and interests of the potential users). They argue that the next generation of intelligent RS will be cognitive

RS, which will comprehend user preferences, recognize changes in those preferences over time, forecast user preferences that are unknown and investigate the processes that will allow intelligent action in rapidly changing situations.

The remaining part of Sect. 2 is structured as follows.

Firstly, we describe two categories of RS, on a recommended-item basis: (i) Academic RS that recommend publications, authors/researchers or venues in Sect. 2.1 and (ii) Generic RS that recommend other types of items (e.g., news, movies, hotels, etc.) in Sect. 2.2.

Next, we present literature which raises concerns in regard to modern RS's true development/progress and criticizes the selected datasets for evaluating their performance (Sect. 2.3).

Moreover, in Sect. 2.4, we refer to the essential preliminaries and theoretical background of some methods and algorithms used in preparation, designing and evaluation of our proposed system (Sect. 3).

2.1 Academic recommender systems

Sakib et al. [8] describe a Collaborative Filtering-based recommendation method for scientific papers that does not rely on user profiles up front and solely makes use of publicly available contextual data. They used 2-level paper-citation links to identify hidden correlations across articles by using the citation context. This strategy is justified by the fact that two papers that are cited in the same article and that appear in the same two papers are highly comparable to each other.

Also, in Sakib et al. [9] state that the main downside of the existing RS approaches is that their effectiveness depends on a priori user profiles, and thus, they cannot recommend papers to the new users (cold start problem). Their proposed system uses both public and non-public metadata, and therefore, the system is unable to find similarities between papers efficiently due to copyright restrictions. In their research, considering the above challenges, a novel hybrid approach is proposed that combines separately a Content-Based Filtering (CBF) recommender module and a Collaborative Filtering (CF) recommender module. Unlike previous CBF and CF approaches, public contextual metadata and paper-citation relationship information are effectively incorporated into these two approaches separately to enhance the recommendation accuracy.

Li and She [10] put forth the collaborative variational autoencoder (CVAE), a Bayesian generative model that takes into account both rating and content for recommendations in multimedia scenarios. The model learns implicit links between items and users from both content and rating, as well as deep latent representations from content data in an unsupervised way. The proposed CVAE, in contrast with earlier efforts with denoising criteria, learns a latent distribution for content in latent space rather than observation space through an inference network and is easily extensible to other multimedia modalities besides text.

In addition, Li and She [11] study link prediction, a vital issue with widespread applications in bioinformatics, information retrieval, social networks and webpage networks. Latent variable models, which jointly model network structure and node properties like the relational topic model and its derivatives, have demonstrated excellent accuracy for predicting network architectures and identifying latent representations among link prediction techniques. However, these methods are still limited in their ability to learn representations from high-dimensional data or solely take into account text as a content modality. They are so severely constrained in the contemporary multimedia environment. They suggest a relational variational autoencoder (RVAE), a Bayesian deep generative model that takes into account both connections and content for link prediction in the multimedia case. The model learns, in an

unsupervised manner, network architectures from both content and link data as well as deep latent representations from content data.

Hsieh et al. [12] propose the Collaborative Metric Learning (CML) which learns a joint metric space to encode not only users' preferences but also the user–user and item–item similarity; they study the connection between metric learning and Collaborative Filtering. The proposed algorithm outperforms state-of-the-art Collaborative Filtering algorithms on a wide range of recommendation tasks and uncovers the underlying spectrum of users' fine-grained preferences. CML also achieves significant speedup for Top-K recommendation tasks using off-the-shelf, approximate nearest-neighbor search, with negligible accuracy reduction.

Moreover, Nikzad-Khasmakhi et al. introduce in [13] the multimodal classification approach for the expert recommendation system (BERTERS). The modalities in the suggested system are obtained from textual (articles written by candidates) and graphical (co-author links) data. BERTERS uses the Bidirectional Encoder Representations Transformer (BERT) to turn text into a vector. Additionally, the co-author network is used to extract the candidate's features using a graph representation method called ExEm. These vectors and other features are concatenated to create the final representation of a candidate. Eventually, a multimodal classifier is constructed based on a combination of features.

Furthermore, Alfarhood and Cheng [2] introduce CATA++: a Collaborative Dual Attentive Autoencoder RS that utilizes Matrix Factorization, CF and learns the user–items latent connections via two discrete Autoencoders which run in parallel. To better represent the items in the latent space, they use the attention mechanism in the middle of the Autoencoders to capture the most important portions of contextual information. To enhance the performance of the recommendations, they used an ANN, matrix factorization and collaborative filtering.

In addition, CATA++ provided better recommendation scores and outperformed RS in the literature, as mentioned in [2] and as it is proved by our experiments [1]. Eger et al. [14] perform a comparison and evaluation of DL activation functions; they argue about the value and importance of the process of HP tuning and the significance of the activation function choice for a DL RS. The limited optimization for a number of Hyper-Parameters (HP) in CATA++, along with the content of [14], gave us the motivation to accomplish HP tuning optimization with a view to improve its performance. We did not alter the architecture or design of CATA++, but we tuned its Hyper-Parameters as described in [1] with significant improvement in the recommendation metrics scores while at the same moment the training phase of the model became faster. To enhance the training and performance of CATA++, we tuned its activation function, weight initialization and training epochs. A range of cutting-edge activation functions have been tested throughout the tests: ReLU [15], LeakyReLU [16] or PReLU [17], ELU [18], SineReLU [19], GELU [20], Mish [21], Swish [22] and Flatten-T Swish [23]. Moreover, many weight initializers have been tested (XavierGlorot, He, Lecun, etc.). We ran experiments testing the effect of changing the training epochs number in the range: 10 to 150. We have ran tests using data from CiteULike website and AMiner Citation Network dataset. The metrics (Recall, NDCG) show that HP-tuning can considerably cut down on training time while improving the performance of recommendations [1].

Furthermore, Kong et al. [24] develop an academic RS, namely VOPRec, by vector representation learning of paper in citation networks. VOPRec takes advantages of recent research in both text and network representation learning for unsupervised feature design. In addition, VOPRec utilizes word embedding to find papers of similar research interest. Then, the structural citation network is converted into citation vectors to retrieve papers of similar citation network topology. Next, VOPRec makes recommendations by combining/bridging word embeddings and structural citation network.

Table 1 Academic recommender systems with their relevant datasets

Paper	RS type	#users	#papers
[8, 9]	CF, Graph	50	100,351 (papers collection)
[2, 10, 11]	CF, DL	5,551	16,980 (citeulike-a)
[2, 10–12]	CF, DL	7,947	25,975 (citeulike-t)
[2]	CF, DL	3,039	210,137 (citeulike2004–2007)
[11]	CF, DL	N/A	29,555 (arxiv part)
[13]	CF, DL	9,378	472,566 (Scopus part)
[24]	CBF, Graph	N/A	7,547 (APS physics)
[25]	CBF, CF, Graph	3,198	3,913 (papers collection)
[26]	Graph	24	4,000 (papers collection)

Tsolakidis et al. [25] present a hybrid RS comprising of: (i) the CBF approach, where the papers published by the authors are indexed and the TF-IDF algorithm is applied to calculate the weights for each one of the indexed terms and (ii) the CF approach, in which authors are considered to prefer similar publications in the future, with authors having similar past behavior. In their proposed system, they contribute to the collaborative filtering by implementing a novel graph-based analysis aiming to define the importance of each indexed term.

Son et al. [26] propose an RS for academic papers that combines citation analysis and network analysis. The proposed method is based on multilevel citation networks that compare all the indirectly linked papers to the paper of interest (POI) to inspect the structural and semantic relationships among them. Their main research objective in this study is to consider the mutual relationships among the papers in a broad network beyond a single level and to evaluate the significance of each paper through certain centrality measures.

The above-mentioned literature is described in Table 1 along with their relevant datasets used for evaluation.

2.2 Generic recommender systems

Kyriakidi et al. [27] make heavy utilization of graphs and argue that most of the recommendation approaches can be translated into a graph exploration problem. In this basis, they describe a theoretical graph framework with two primary parts: (a) a recommendation graph, which models all the elements of a domain (subjects and objects of recommendations) as well as the relationships; (b) a set of path operations, inferring new edges, by finding implicit, latent and undiscovered relationships.

Shambour [28] proposes a Deep Learning-based architecture with a view to create a multi-criteria RS in which deep Autoencoders exploit the hidden relations between users with regard to multi-criteria preferences, and generate recommendations with higher accuracy scores.

Shi et al. [29] study distributed and reinforcement learning RS, so they propose DARES, a distributed RS algorithm that uses reinforcement learning and is based on the asynchronous advantage actor-critic model (A3C). DARES is developed combining the approaches of A3C and federated learning (FL) and allows users to keep their data locally on their own devices. The system architecture consists of (i) a local recommendation model trained locally on the

Table 2 Generic recommender systems with their datasets

Paper	RS type	#users	#items
[27]	Graph	1,6 M	193K (Yelp Businesses)
[28]	CF, DL	1,716	965 (Yahoo! Movies)
		1039	693 (TripAdvisor Hotels)
		13,158	731 (Smart-Media News Adressa)
[29]	CF, DL	22,713	2,302 (Outbrain News)
		943	1,682 (ML100k Movies)
		10,000	7,000 (Netflix-subset Movies)
[30]	CF, DL	6,040	3,706 (MovieLens Movies)
		1,227	2,509 (FilmTrust Movies)
		4,902	1,683 (Instant Video)
[31]	CF	1,310	1,635 (IMDB Movies)
		1,631	1,631 (Yelp Businesses)

user devices using their interaction and (ii) a global recommendation model that is trained on a central server using the model updates that are computed on the user devices.

Bobadilla et al. [30] propose a cutting-edge DL architecture to enhance CF outcomes in RS. It takes advantage of the dependability concept's promise to improve the accuracy of predictions and advice by adding prediction mistakes (reliabilities) into the DL layers. The fundamental aim is to endorse highly anticipated products that have also been validated as trustworthy products. The nonlinear relationships between forecasts, reliabilities and precise recommendations are extracted using the DL architecture. Real prediction errors, predicted errors (reliabilities) and predicted ratings (predictions) are the three connected steps that make up the proposed architecture. They utilize two techniques: Matrix Factorization for the ratings, Multilayer Neural Network fed with reliabilities and hidden factors.

Li et al. [31] present the Symmetric Metric Learning (SML) algorithm with a new proposed feature of adaptive and variable margins. Metric learning-based methods have been utilized extensively in the literature's RS, but current methods apply a user-centric way in space, to ensure the distance between the user and a negative item to be larger than that between the current user and a positive item by a fixed margin. At the same time, they ignore the relations among positive items and negative items. If these two items are positioned closely, the RS may produce incorrect results (bad recommendations). Meanwhile, different users usually have different preferences, so the fixed margin used in current methods cannot be retrieved and used to various user biases; thus, in this case too, the fixed margin decreases the recommendation performance as well.

The above-mentioned literature is described in Table 2 along with their relevant datasets used for evaluation.

2.3 Concerns and critique

Dacrema et al. [32] raise concerns about the Machine Learning RS's performance and progress in the recent years. Deep Learning-based methodologies, in particular, are currently occupying center stage in the literature. All of them are said to have made significant advancements over current best practices. However, there are signs of certain issues with current research practices, such as the selection and improvement in the baselines used for

Table 3 CiteULike website available datasets

Dataset	#users	#papers
citeulike-a	5K	17K
citeulike-t	8K	26K
citeulike-2004-2007	3K	210K

comparison, which cast doubt on the published claims. They have contrasted current findings in the field of neural recommendation methods based on collaborative filtering against a uniform collection of simple baselines in order to better grasp the actual improvement. They found that 11 of the 12 reproducible neural approaches could be outperformed by technically straightforward methods, such as those based on the nearest-neighbor heuristic or linear models. These recent works, all of which were published between 2015 and 2018, were all presented at highly esteemed scientific conferences. None of the computationally demanding neural methods consistently outperformed state-of-the-art learning-based methods, such as linear models or matrix factorization. In their discussion, they address prevalent problems in current research practices that, despite the large number of articles published on the subject, appear to have caused some amount of stagnation in the field.

Moreover, Dacrema et al. [32] critique the selection of baselines and observe that, occasionally, researchers repurpose experimental designs from earlier studies, utilizing the same datasets, evaluating procedures and measures to show advancement. This indicates that the experimental design is no longer under scrutiny because it has been employed by numerous researchers in the past. Some of the articles in their analysis, according to the authors, were based on experiments using small-scale CiteULike datasets (Table 3) that were put to use in some earlier Deep Learning publications. Later, additional scientists reused these CiteULike datasets without considering their usefulness for studying particular phenomena or whether they were indicative of a larger range of actual recommendation difficulties.

Added to that, Da Silva et al. [33] raise similar concerns regarding the datasets' size used in various RS approaches in recent literature, the evaluation tactics and the reproducibility restrictions that are very common for many RS.

Azeroual and Koltay [34] state that CF methods have potential scaling problems due to the memory-expensive Matrix Factorization algorithm that has to be carried out on increasingly large datasets.

As it is pointed out by Dacrema et al. [32], Da Silva et al. [33] and the data in Tables 1 and 2, the magnitude of these testing datasets is rather undersized in comparison to the online digital libraries available today in various categories, such as movies, social data, scientific publications, etc. Therefore, moderately speaking, as far as the dataset's size is concerned, an apparent mismatch exists between state-of-the-art and real-world problems or real-world datasets (millions of users-items scale). This problem has been the main motivation of this work; we focus on trying to counterpoise the state-of-the-art RS approaches with real-world problems and (large) real-world datasets.

2.4 Background

Dimensionality reduction techniques

Feature selection (FS) or dimensionality reduction (DR) is simply a process that reduces the number of input variables, in order to keep only the most important ones. There is an advantage in reducing the number of input features, as it simplifies the model, reduces the

computation cost and can also improve the model's performance. This process is absolutely necessary in academic RS as the textual data they need to utilize and process are highly dimensional data; each unique word defines a different feature for the system.

In regard to text vectorization, we adopt the TF-IDF method, a statistical technique that is frequently used in information retrieval and natural language processing. It gauges a term's significance inside an article in relation to a corpus, or group, of documents. A text vectorization procedure converts words in a text document into significance numbers. There are numerous distinct scoring methods for text vectorization, with TF-IDF being among the most used. The TF-IDF vectorization/scoring method, as the name suggests, multiplies the Term Frequency (TF) and Inverse Document Frequency (IDF) of a word to determine its score.

Meijer et al. [35] focus on the performance (quality and efficiency) of word embeddings in comparison with TF-IDF representations while modeling the content of scientific articles. They evaluated the two algorithms on a categorization task that matches papers to journals for about 1.3 million papers. The results show that content models based on word embeddings are better for titles (short text), while TF-IDF works better for abstracts (longer text). Since in our system we utilize and combine both title and abstract of papers, we need to process rather long text, and therefore, TF-IDF is the most suitable technique.

Term Frequency: TF of a term or word is the number of times the term appears in a document compared to the total number of words in the document.

Inverse Document Frequency: IDF of a term reflects the proportion of documents in the corpus that contain the term. Words unique to a small percentage of documents (e.g., technical terms) receive higher importance values than words common across all documents (e.g., a, the, and).

Term Frequency (TF) measures how frequently a term or word comes up in a document relative to all the other words in it.

The percentage of articles in the corpus that include a phrase is indicated by its inverse document frequency (IDF). Technical terminology and other words that are specific to a tiny percentage of documents are given greater relevance ratings than words that are used in all papers, such as the, you, and, etc.

Equation (1) is the typical mathematical formula of TF-IDF.

$$w_{i,j} = tf_{i,j} \times \log \frac{N}{df_i} \quad (1)$$

where $w_{i,j}$ is the weight (TF-IDF) for the term i in the document j , N is the number of documents in the collection, $tf_{i,j}$ is the term frequency of the term i in document j and df_i is the document frequency of term i in the collection.

As soon as the TF-IDF calculations for all terms are finished and in order to complete the feature selection process. The approach ignores terms with a strict document frequency above the specified threshold while constructing the vocabulary (corpus-specific stop words).

The elbow method

Prior to performing K-means clustering, it is utilized to find the ideal number k of centroids or clusters. By using various choices of k , the elbow approach plots the estimated value of the cost function. Each cluster will have fewer constituent instances and be closer to its own centroids as k rises, resulting in a drop in the average distortion. However, once k exceeds a certain level, the improvements in mean distortion will start to drop. The value of the parameter k where the improvement in distortion score declines the most is known as the

elbow; at this point we should stop increasing the clusters' number by dividing further our data.

Silhouette Coefficient or silhouette score is a metric used to calculate the quality of the clustering. Its value ranges from -1 to 1 .

1: Clusters are well apart from each other and clearly distinguished.

0: Clusters are indifferent, or we can say that the distance between clusters is not significant.

-1 : Clusters are assigned in the wrong way.

The Silhouette score is calculated mathematically as:

$$\text{Silhouette Score} = \frac{(b - a)}{\max(a, b)} \quad (2)$$

where:

a equals the average intra-cluster distance (i.e., the mean value of the distances between each point within a cluster);

b equals the average inter-cluster distance (i.e., the mean value of the distances between all clusters).

So, the optimal value for the Silhouette score should be near or equal to 1 , in order to have clusters well apart from each other and clearly distinguished.

Evaluation metrics

Recall & Normalized Discounted Cumulative Gain (NDCG) are two measures used to assess the effectiveness of our trials.

Recall (Sensitivity or True Positive Rate) is the ratio of the number of pertinent documents returned by a search to the total number of pertinent documents already in existence. Recall does not, however, assess the quality of the rankings inside the top-K list. As a result, NDCG is used to demonstrate a model's capacity to suggest articles for the top of the list of suggestions; in reality, it is a gauge of ranking quality. Recall & NDCG metrics are calculated mathematically as described in [2], while extensive description of these metrics can be found in our previous work [1].

3 Proposed system

This section presents a detailed description of our proposed system.

We build a multi-stage, hybrid system, based on a weighted-graph model, where clustering techniques are exploited. Content-based Filtering (CBF) is applied at the first stages of this process, while in the last stage, a Machine Learning (Artificial Neural Network—Deep Learning), Collaborative Filtering (CF) RS is incorporated. This way we are able to increase the amount of scientific publications the system exploits and overcome issues like cold start (for a new user or publication). Also, our system deals with the fact that ANN models (especially Deep Learning -DL- architectures) are highly intensive regarding memory, CPU or GPU, during the training process. Usually state-of-the-art RS that use an ANN/DL architecture while running on a single machine (PC or Server) can exploit some thousands of papers in order to train a model and make recommendations, due to hardware limitations (memory, CPU, or GPU). Therefore, we propose a multi-stage, hybrid (Graph, Cluster, ANN, CBF, CF) RS that exploits various techniques and manages to run on a dataset of millions of scientific publications.

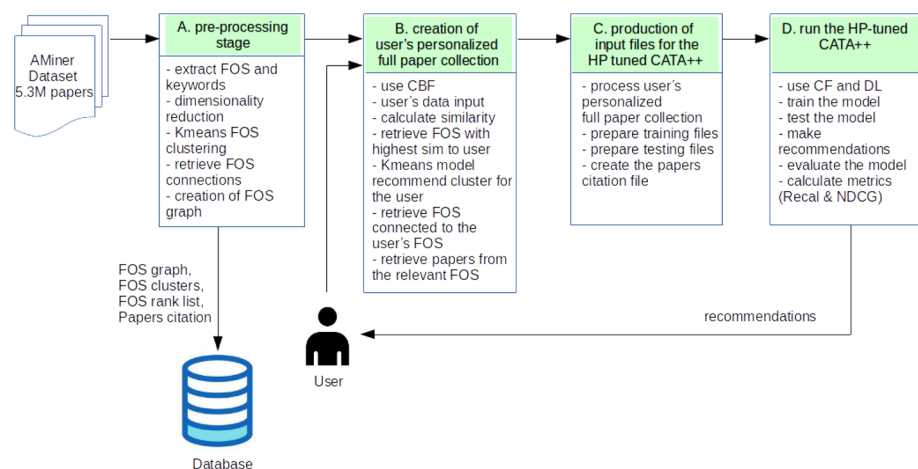


Fig. 1 Proposed system architecture

The following are the significant contributions we made:

- (1) We propose a novel Graph-model, Clustering, multi-staged, ANN RS, which is also hybrid (CBF and CF), as depicted in Fig. 1.
- (2) We manage to exploit and make recommendations on the full DBLP AMiner Citation Network dataset, containing about 5.3M papers, whereas other pure ANN or DL approaches can use only some thousands of it.
- (3) We perform clustering of the scientific publications found in the DBLP AMiner Citation Network dataset, on a *field of study* (fos) similarity basis.
- (4) We create a weighted, non-directed graph of the fos described on the above-mentioned dataset.
- (5) We embody an ANN/DL RS, after tuning its Hyper-Parameters and improving training and performance, which uses Autoencoder (ANN), Matrix Factorization and Collaborative Filtering.

To accomplish the above steps, our multi-staged system (Fig. 1) is divided into four discrete stages:

- Stage A Sect. 3.1: Pre-processing of the full DBLP AMiner dataset and producing the graph-model and the clustering based on the fos of papers.
- Stage B Sect. 3.2: Creation of the user's personalized full paper collection, through the user's input.
- Stage C Sect. 3.3: Production of the necessary input files for the HP-tuned CATA++ (train and test files).
- Stage D Sect. 3.4: Running the HP-tuned CATA++ [1]; make recommendations and evaluate the system using Recall and NDCG.

Implementation

Our academic RS is implemented using the Python programming language and the PyCharm Professional IDE. Several Python Machine Learning libraries are used, such as Tensorflow, Keras, NLTK, NumPy, SciPy and scikit-learn. In a few hours, a typical modern computer (Ubuntu 18 OS, CPU Intel Core i5, RAM 16GB) can complete the system's extensive computing and preprocessing tasks. Moreover, the system's database is a standalone SQLite

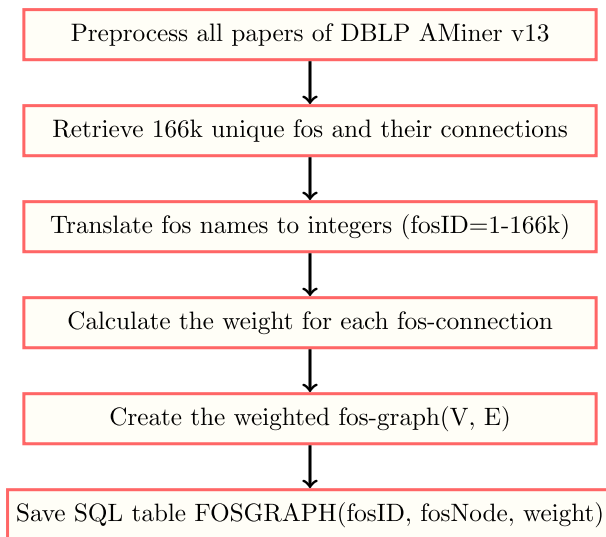


Fig. 2 Stage A (A1). Creating the weighted FOS-GRAPH($V=fos$, $E=connections$)

database, while in some cases the Python Pickle module is used, which is another way to save the current state of objects in external files.

Dataset

All the development and experiments of our system were made based on data from the DBLP-Citation-network-v13 (2021-05-14)¹ from AMiner [36] which was available at the time this work was written, and which included 48,227,950 citation relationships and 5,354,309 publications.

DBLP-Citation-network-v13 is available in json format and consists of a variety of information for each paper, while some of the most useful are: paper's unique number, title, author, keywords, paper fields of study (fos), abstract or indexed abstract.

3.1 Stage A. Pre-processing of the DBLP AMiner citation network dataset

The algorithm starts with the offline pre-processing stage of the DBLP-Citation-network-v13.

This offline stage runs only once for the entire dataset before users log in to the system. It produces two major outputs: the weighted, non-directed fos-graph (described in A1 stage 3.1.1) and the fos-clustering (described in A2 stage 3.1.2).

To begin with, the original idea of our approach is to use the field of study (fos) that the papers belong to, rather than the actual papers, since their number (about 5.3M) becomes a limiting factor. The number of unique fos described in the dataset rises to 166K, a number that is feasible for a system to work with.

3.1.1 Stage A1. Building the FOS-graph

Our algorithm, described in Fig. 2, analyzes the available information for each paper and creates a weighted, non-directed graph as depicted in Fig. 3.

¹ <https://www.aminer.org/citation>.

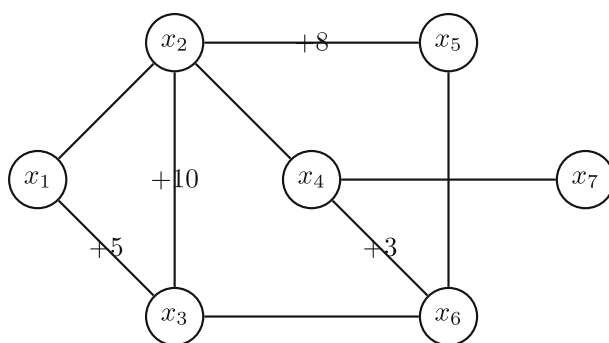


Fig. 3 Field of study, weighted, non-directed graph

A vertex (or node) of the graph is a unique fos, and it may be connected to one or more other fos with a weighted edge. Various fos appear in Fig. 3 like the vertices: x_1 , x_2 , x_3 , etc.

An edge connects two fos when they appear in the fos-list of a single paper. When these two fos appear in more papers, the weight of the edge increases accordingly. As mentioned before, in the AMiner Citation Network dataset, each paper provides a list of fos in which it belongs; i.e., for a random paper p_1 the fos-list may be: ["Mobile network operator", "Wireless network", "Mathematical proof", "Wireless", "Correlation and dependence"]. So, from paper p_1 will derive 5 nodes: n_1 (Mobile network operator), n_2 (Wireless network), n_3 (Mathematical proof), n_4 (Wireless), n_5 (Correlation and dependence), and the relevant edges that connect these nodes:

- edge #1 (n_1 - n_2), edge #2 (n_1 - n_3), edge #3 (n_1 - n_4), edge #4 (n_1 - n_5)
- edge #5 (n_2 - n_3), edge #6 (n_2 - n_4), edge #7 (n_2 - n_5)
- edge #8 (n_3 - n_4), edge #9 (n_3 - n_5)
- edge #10 (n_4 - n_5)

The weight on the edges shows how many times this specific fos-couple has been found in papers. If the edge does not have a number, it means that the weight equals one.

This way we manage to organize different fos into a weighted graph depending on how many times those fos have been found together in papers. So, in a fos-graph area (or neighborhood) we can find fos that belong in the same, relevant or close scientific areas. Later the system, as soon as it has classified the user to some fos, it can retrieve the fos of the neighborhood and use it to retrieve papers for recommendation.

Moreover, using the fos-graph, we create a descending order, sorted list of fos on a basis of total connections' weight. The total weight is calculated by adding each edge's weight for a single fos (node). This way we discover the most important fos (those with the most connections to other fos), since they will be at the first indexes (top) of the fos list. Thus, the system adds a bias to those very important fos (like center of gravity) for the graph.

The fos-graph allows the system to easily retrieve knowledge about relationships between nodes. In addition, the fos-graph is useful to describe the strongly connected entities (fos). Using the fos-graph, we can discover relationships or knowledge paths that connect fos in scientific areas that otherwise is very difficult to find. The system is using this latent, hidden paths of fos, to increase the diversity of recommended items.

The fos-graph also helps the system to overcome the cold start problem, because even the most recently published papers are classified in one or more fos and this way they become part of our graph. The paper's publication year or received citations are not part of this graph.

Moreover, the use of the fos-graph helps to overcome the sparsity problem, when little or no user behavioral data are available for a user or paper; it is built irrelevantly to the number of times a paper has been read or liked.

3.1.2 Stage A2. Building the fos-clustering

Moreover, in stage A the fos-clustering is produced, as a result of the K-means clustering method in Fig. 4.

Firstly, we create a *bag of words* (*bow*), which holds a list of keywords, describing each fos; for example, there is a random paper p1 with the fos-list: ["Mobile network operator", "Wireless network", "Mathematical proof", "Wireless", "Correlation and dependence"] and another paper p2 with the fos-list: ["Analytic proof", "Algebra", "Structural proof theory", "Mathematical proof", "Proof complexity"]. The fos "Mobile network operator" will have as a description the text from the title and abstract of p1, but the fos "Mathematical proof" which is found in both papers' fos-list, will have as description the text from both papers. So, each fos is described by the available text (title and abstract) of all papers that it can be found. Afterward, we retrieve the keywords or bow from this description (plain text) for each fos.

The distance between two fos is calculated on a basis of keywords or bow similarity between those fos. When two fos have a high bow similarity, they have a small distance between them, so they are going to be allocated to the same cluster.

For each fos, we had to vectorize its bow, because bows are highly dimensional textual data; in order to accomplish that, we used the TF-IDF method.

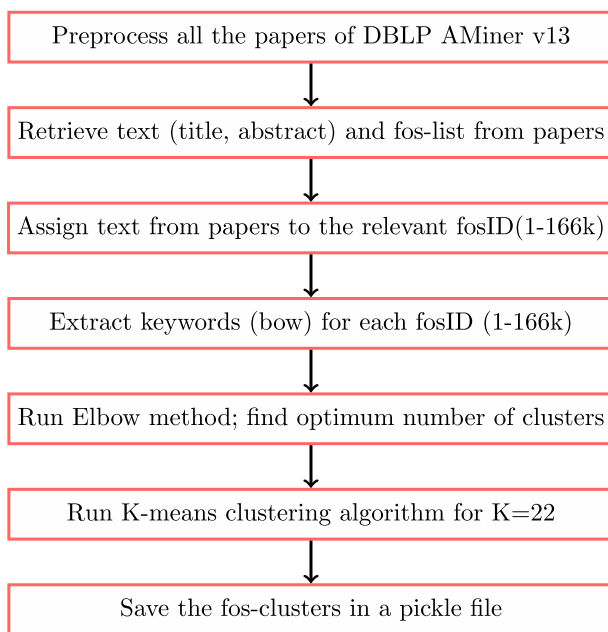


Fig. 4 Stage A (A2). fos-clustering with K-means, where the distance among different fos is the bow (key-words) similarity pairwise

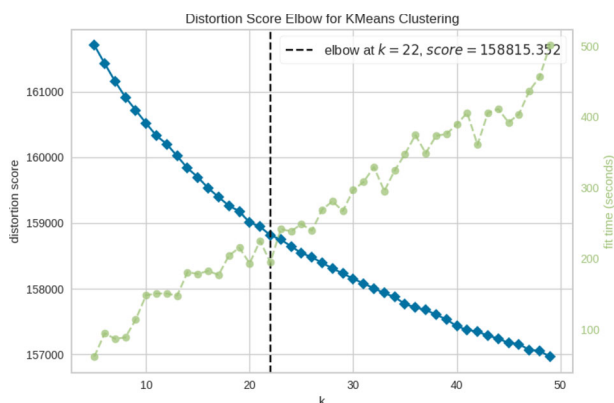


Fig. 5 Distortion score of Elbow method

In our work, we set the *TF-IDF upper threshold* parameter equal to 0.6; this means that features (keywords) appearing in more than 60% of the 5.3M papers are discarded because they are extremely common and do not add significant value to the clustering, but only computational cost. The *TF-IDF lower threshold* parameter, which is a float or integer number (default=1), should also be set. When building the vocabulary of the keywords, the method rejects words that have a document frequency lower than the given threshold; this value is also called cut-off. If *TF-IDF lower threshold* is a float in the range of [0.0, 1.0], the parameter represents a fraction or percentage of documents, while if it is an integer, it represents the absolute counts. We set the *TF-IDF lower threshold* equal to 10, which makes the features (keywords) that appear in 10 or less papers to be discarded, because they are considered irrelevant to the subject of the dataset (computer science).

Moreover, we need to implement some feature selection (dimensionality reduction); we use the truncated singular value decomposition (TSVD) method, which is a dimensionality reduction technique. This transformer performs linear dimensionality reduction by utilizing the singular value decomposition method. Unlike the Principal Component Analysis (PCA), TSVD estimator does not center the data before calculating the singular value decomposition. This provides TSVD with the capability of being able to process sparse matrices efficiently (while PCA cannot). The *n components* parameter of TSVD is an integer, which defaults to 2, that defines the desired dimensionality of the output data. The default value of 2 is useful only for visualization purposes. We use this dimensionality reduction technique in order to produce the result of Fig. 7.

Define the optimal number *k* of centroids

In order to define the optimal number *k* of centroids/clusters, we used the elbow method (distortion score) and silhouette score; details are discussed earlier, in the *elbow method* paragraph 2.4.

Initially, we run the *elbow algorithm* to determine the ideal number of *k* (centroids' and clusters' number). The optimal number of centroids *k* is found to be 22 in accordance with (iaw) the elbow method execution's plot in Fig. 5.

As pointed out earlier, the optimal value for the Silhouette score should be near or equal to 1, in order to have clusters well apart from each other and clearly distinguished. Also, it is very important for all the clusters to have values bigger than the mean value; all the clusters should peak over and right of the red dashed line (mean Silhouette score). In Fig. 6,

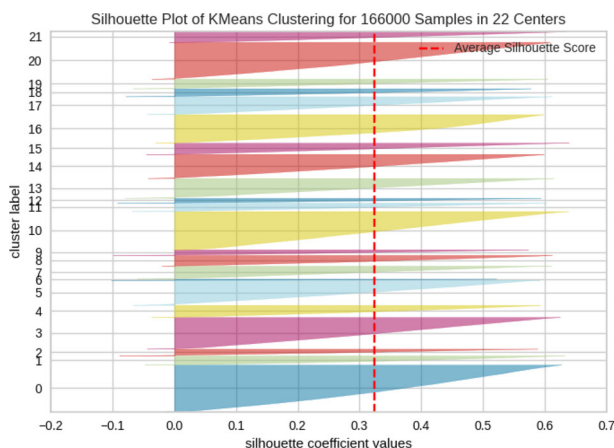


Fig. 6 Silhouette score for K=22 centroids

we can see the silhouette score value, while we are trying to cluster 166k fos in 22 centroids; it presents a mean value of 0.32, which remains the same for a number of centroids in the range of [20, 28]. During experiments with $k > 28$, the mean score is getting lower as the centroids' number increases. Moreover, in Fig. 6 all the clusters have a peak over and right of the red dashed line (mean value).

Therefore, combining the results of the elbow method's distortion and silhouette scores, we reached the conclusion of using $k=22$ centroids/clusters.

Run the clustering algorithm

We performed various experiments with K-means and minibatch K-means, in order to accomplish the clustering of the 166k unique fos. We tried minibatch K-means to speed up the process, but the resulting clusters produced by K-means and minibatch K-means were very similar and with similar running time. So, we decided to use the K-means algorithm resulting clusters.

In all clustering experiments, we enabled the *K-means++* centroids initialization feature of the K-means method, which, based on an empirical probability distribution of the points' contributions to the total inertia, uses sampling to choose the initial cluster centroids. This method hastens the model's convergence. The algorithm used is *greedy K-means++*, which makes several tests at each sampling stage and this way selecting the best centroid across the samples.

Finally, the algorithm clusters 166k total fos around 22 centroids; the size of each cluster is presented in Table 4, while the visualization (plot) of the clusters is shown in Fig. 7 (the dots depict the position of the 22 centroids for the relevant clusters).

3.2 Stage B. Creation of the user's personalized paper collection

This stage of the system takes as input the products of the dataset's pre-processing stage (fos-clustering and FOS-Graph saved in a sqlite database)—as well as the user's input (may include: text of interest/list of liked papers/publication year/digital object identifier (DOI)/text from papers he or she likes).

The steps of Stage B are recorded concisely in Algorithm 1.

Table 4 Clusters' size (total fos per cluster in descending order)

Cluster Id	Total fos
2	24810
1	15756
5	10756
7	10480
17	9288
21	7944
3	7807
0	7198
9	7069
10	6904
11	6852
8	6437
4	6356
15	5889
13	5785
12	5661
20	5071
6	4039
19	3906
18	3256
14	3080
16	1656

Stage B outputs the personalized recommended paper collection for the user, about 30K papers maximum due to hardware limitations (CPU, RAM, GPU) of a medium modern PC.

The steps of the above-mentioned algorithm are described below; three of these steps have different versions in order to produce three different versions of our system. These steps were selected because they make a major contribution in the process of creating the full paper collection for the user; also, they provide some different characteristics in our system, i.e., recommendation diversity and mitigating the cold start problem.

Firstly, the user provides the necessary input: (i) text of interest (keywords/abstract of paper/full paper, etc.) or/and a list of liked papers (list of DOIs) and (ii) the start of a publication period that he or she is interested in. From the user's text or list of liked papers, the system extracts the user's keywords (bag of words—bow), while the publication year is used to filter out the older papers. The list of liked papers, if provided, is utilized in the collaborative filtering method of the last stage of our system. This input step (text or list of papers) of Stage B emulates the procedure of a user that browses in the RS's papers and provides direct feedback through clicking, reading, buying or rating papers that would provide the necessary information for the collaborative filtering and the content-based filtering.

There is also an optional feature, where the user is prompted to provide some extra text (titles, abstracts, full papers, etc.) of papers he/she likes. In case extra text is provided, the system will extract more keywords and them to the bow created previously.

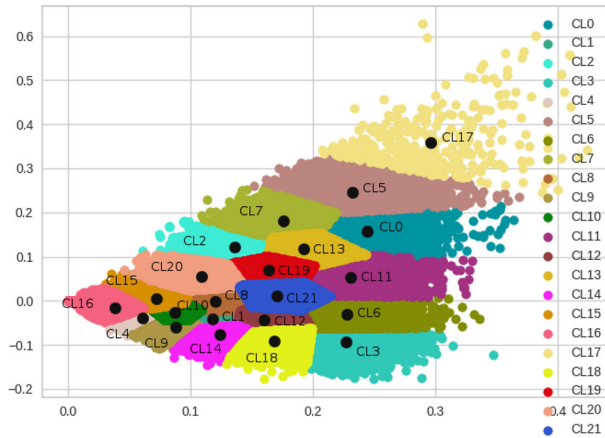


Fig. 7 Plot of K-means fos-clustering; K=22 centroids (black dots)

Algorithm 1 Stage B. Creation of the user's personalized paper collection

Require: The products of AMiner dataset's pre-processing & user's input

Ensure: The personalized paper collection for the user

```

1: take user input
2: if DOI is provided then
3:   save papers to paper collection & retrieve fos of these papers
4: end if
5: if text from papers is provided then
6:   keywords extraction
7: end if
8: K-means model cluster recommendation for the user
9: for fos in the recommended cluster do
10:  calculate cosine similarity: user's keywords & fos bow
11: end for
12: sort fos in similarity descending order & retrieve fos over the threshold
13: for fos in expanded fos-list do
14:  retrieve all papers and add them to the full paper collection
15: end for
16: filter the full paper collection with the publication year
17: while total Papers  $\leq 40,000$  do
18:  add papers from various fos-lists
19: end while
20: while total Papers  $\leq 30,000$  do
21:  if highest cited bias is ON then
22:    add highest cited papers to the final paper collection
23:  else
24:    add most relevant papers to the final paper collection
25:  end if
26: end while
27: return the personalized paper collection for the user

```

Using the K-means model that was created at Stage A, the system makes a cluster recommendation for the keywords assigned to the user; i.e., the system assigns the user's keywords (bow) to a fos-cluster.

The system calculates the cosine similarity value on pairwise basis between the user input keywords and every fos in the recommended cluster (all fos already have a bow—list

of keywords—from Stage A) and accumulates the best similarity fos-list for the user. The system rejects fos with similarity equal to zero; then, it retrieves the fos with top similarity, to create the “best similarity fos-list”, in one of the following ways:

- *1st version* The system retrieves the fos with the top 10% similarity (starting point) and increases the percentage until the fos-list reaches the maximum threshold of 300 fos; the parameter *top similarity percentage* is set initially to 0.1, increases gradually and never exceeds 0.6. This way the 1st version of our RS keeps the fos-list quite small (max 300 fos), by holding the most similar fos to the user’s keywords. Also, the fos-list is characterized by great similarity and coherence, while the diversity is kept low; as a result, the fos that describe scientific areas around or nearby the user’s interests are not added to the list, so the user will not see papers of those fos as a recommendation.
- *2nd version* The system always retrieves the fos in the top 20% similarity (constant parameter with a value of 0.2); this setting has a direct result to the fos-list’s size which varies from 500 to 1,800 fos. The exact number varies in accordance with (iaw) the size of the cluster that the user has been allocated to (see Table 4). This way we give the fos-list a level of diversity and lower similarity among the included fos. As a result, the fos-list contains some fos that are not in the user’s initial interests, but they represent very close or neighboring scientific areas that might possibly interest the user. This 2nd version is provided with the capability to create a sphere (complex) of scientific areas or subjects that comes (more or less) to the user’s interest but always have his/her primary likes in the center.
- *3rd version* The *top similarity percentage* is selected in accordance with the recommended cluster’s size (see Table 4); the bigger the cluster, the smaller similarity percentage we select to use; this way we always gather for the user about 2000 fos from the recommended cluster to add to the fos-list. As a result, the fos-list holds a bigger number of fos, with higher diversity and many close or nearby scientific areas to the first (main) user’s area of interest. The sphere of scientific areas or subjects that come (more or less) to the user’s interest is now even bigger, providing the user the opportunity to see diverse recommendations.

In the next step, we create the “expanded fos-list” as the intersection of two sets (see Fig. 8):

- Using the FOS-Graph, we retrieve the fos that are connected with more than one fos of the best similarity fos-list (created in the previous step). In this way, we create the “fos-list connections” set.
- The set of fos that belong to the recommended cluster (for the specific user).

In the 3rd version of our system, there is an additional step; fos of the user liked papers are extracted and their connections are retrieved by the fos-graph. Finally, we add all these fos (retrieved and their connections) to the expanded fos-list (see Fig. 9).

At this point, the system changes the focus from the fos to the actual papers. In order to achieve that we utilize the best similarity fos-list, the expanded fos-list, the cluster’s top fos-list and we retrieve all the papers that belong in those fos; this way the full personalized paper collection for the user is created.

The system filters the full paper collection with the publication year the user has provided.

The system selects papers for the final paper collection with the following sequence: papers of the best similarity fos-list, papers of the expanded best similarity fos-list and finally papers of the cluster’s top fos-list. In this step, the algorithm stops adding papers when the threshold of 40k papers is reached.

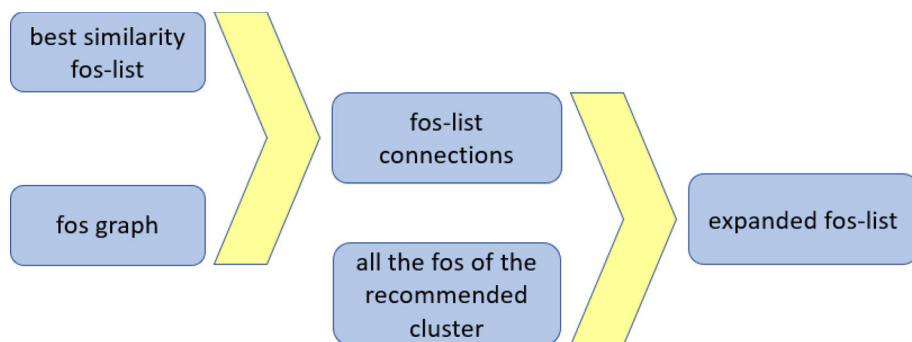


Fig. 8 Creation of the expanded fos-list (intersection of fos-list connections and the recommended cluster's fos)

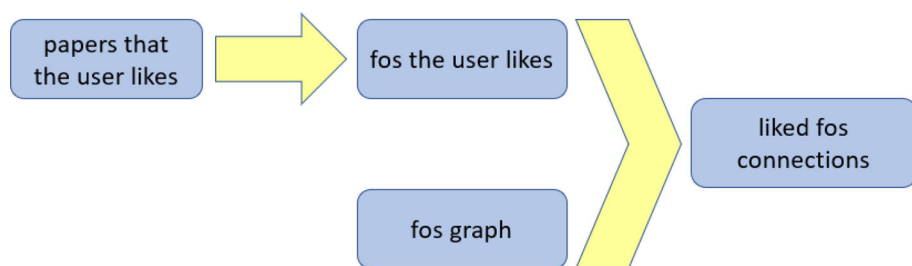


Fig. 9 Creation of “liked fos connections set” as an extra step of our system's 3rd version

Table 5 The three versions of our system's Stage B

RS version	Top similarity percentage	Best similarity fos-connections	Highest citation bias
Ver. 1	Start at 10% and increase (300 fos)	Expanded fos-list	On
Ver. 2	Constant 20% (500–1800 fos)	Expanded fos-list	Off
Ver. 3	Law cluster's size (2000 fos)	Expanded fos-list plus the user's liked fos	On

Finally, we filter the previously created paper collection using the papers' citations to retrieve the highest cited papers and create the final paper collection for the user (30k papers maximum). This step adds bias to the highest cited papers (not taking into account a very recently published paper). In the 2nd version of our system, we remove the bias to the highest cited papers, while the 1st and 3rd versions are biased to the highest cited papers. So, the 2nd version is more likely to recommend even very recent publications, irrelevantly to having been cited or not. This way we make the 2nd version more efficient in cold start cases (new publications), being able to recommend the most recent papers, even if they have not been cited yet.

The differences of the Stage B versions are presented overall in Table 5.

3.3 Stage C. Production of the input files for HP-tuned CATA++

Stage C takes as input the personalized paper collection that was created previously for the user (30k papers max) and produces the necessary input files for the last stage of our system.

The explicit rating values that users give products represent data from the real interactions between users and those items. However, user-provided explicit ratings are typically rare and scarce, which renders the CF mechanism ineffective in some situations (data sparsity and cold start problem).

In our scenario, there are no ratings connecting the authors of articles and the actual papers in the dataset, we have access to (AMiner Citation Network v13). Implicit interaction data across users and items are used to reduce the problem of sparsity. In general, assumptions are used to acquire the implicit interaction data. Here, it is assumed that academic paper citation and reference practices resemble explicit rating practices. A reference implies an implicit, advantageous user–item interaction, so long as it is a deliberate activity on the part of the author. In this manner, the user–item interaction matrix is created, with a user citing a document receiving a positive score (one) or a score of zero (when the user is either unaware of this paper or dislikes it).

In Stage C, we create the set of files required by CATA++:

1. citations: references between publications
2. tag item: publications that contain each keyword
3. users: contains the publications that the users like
4. MATLAB file: the vectorized weights of keywords
5. train & test files

The user's whole collection of papers is also subject to data pre-processing in this step because all data (the content of the papers) consist of textual information that an RS cannot utilize in its raw form. Consequently, a data preparation system is required. The user's paper collection serves as the input, and many processes are taken to pre-process the data:

- (1) Text to lowercase
- (2) Removal of punctuation & numerical data
- (3) Removal of English Language stop words
- (4) Stemming for all keywords
- (5) Delete extremely short/long words
- (6) Removal of unrelated words
- (7) Delete extremely rare keywords
- (8) Run TF-IDF method

The “removal of unrelated words” step in the recommendation process involves removing words that do not provide any pertinent information to the area of computer science. The phrase “unrelated words” contains these words. Our system was initially run several times, and after a semi-automated examination of the keywords obtained from the abstract and title of each document, we realized that certain terms were irrelevant and unrelated to the subject matter (Computer Science in general), we built a “Unrelated words list” (like a blacklist of terms that did not contain any valuable information). Since AMiner serves as an academic bibliography for computer science, we chose to eliminate those terms that had no connection to the topic. This removal stage accomplished information-preserving data reduction while generating the input files required by the RS in the following step (HP-tuned CATA++). This stage eased a significant amount of workload from the total recommendation process. We discuss our strategy, practices and data pretreatment techniques in [37]. The experimental

findings show the effectiveness of our method, with at least 79.8% of the data reduction keeping information.

After applying the pre-processing step (Stage C), as described in our previous work [37], we are able to provide appropriate input files to our system, so that it can answer other recommendation questions, besides academic recommendation, e.g., collaboration recommendation or publication venue recommendation. Details can be found in [37].

3.4 Stage D. Running HP-tuned CATA++

In stage D, we utilize embody the Hyper-Parameters-tuned (HP-tuned) version [1] of a Machine Learning (ML) RS, namely CATA++ [2].

Alfarhood and Cheng [2] describe the Collaborative Dual Attentive Autoencoder (CATA++) RS technique, which makes use of the content of an item to train two concurrent Autoencoders to learn its latent space. To better represent the items in the latent space, they use the attention mechanism in the middle of the Autoencoders to capture the most important portions of contextual information. To enhance the performance of the recommendations, they used an ANN, collaborative filtering and matrix factorization.

The lack of specific HP optimization in CATA++ original system inspired us to use HP tuning to improve the performance of its recommendations. To reduce training time and boost recommendation performance, we examined and tuned a number of cutting-edge activation functions, weight initialization and training epochs. It has to be noted that we did not change the CATA++ format and architecture.

Extensive experiments were conducted using the datasets in Table 6, for three different recommendation questions: scientific publications, publishing venue and collaboration. The analysis of the results provided useful knowledge and guidance, with a view to formulate the HP-tuned variants of CATA++ (see Table 7).

Extended details and experiments on the work of HP-tuning related to CATA++ can be found in [1].

Since HP-tuned CATA++ RS performed better than rest RS of the literature, we decided to employ it in this work, as the stage D of the proposed system.

Our results indicate that He-Normal and Glorot-Uniform weight initializations offer the highest level of performance, delivering an improvement of more than 2% in the citeulike-a dataset trials.

The top-3 Recall results for each test of the CATA++ tuned variants are presented in Table 8, along with the improvement (maximized for the second dataset because its size improved training) in comparison with the default CATA++.

One of our key results is that SineReLU is a highly stable activation function that consistently performs among the top 3 in Table 8 across a variety of datasets. FTS, which

Table 6 Datasets

Dataset	#users	#items	#tags	#sparsity	Recommendation type
citeulike-a	5551	16980	7386	99.78%	Scientific publications
dblp13_collection1	6959	27025	6137	99.95%	Scientific publications
dblp13_venues	14687	2389	7187	99.92%	Publication venues
dblp13_people	16213	21630	8144	99.99%	People (collaboration)

Table 7 HP-tuned versions of CATA++

CATA++ version	Activation	Weight init	Training epochs
FTS-heN-150	Flatten-T Swish	He (Normal Distribution)	150
Swish-gloU-xx	Swish	Glorot (Uniform Distribution)	40–80
Mish-heN-xx	Mish	He (Normal Distribution)	40–80
GELU-gloU-xx	GELU	Glorot (Uniform Distribution)	40–80
SReLU-heN-xx	SineReLU	He (Normal Distribution)	40–80
ELU-gloU-xx	ELU	Glorot (Uniform Distribution)	40–80
LReLU-heN-xx	LeakyReLU	He (Normal Distribution)	40–80

Table 8 Top-3 recall performances for different experiments (datasets)

experiment-dataset	CATA++ version (Top-3)	Improvement
citeulike-a	FTS-150	2%
	Default CATA++	0
	GELU-40 & SReLU-60	0
dblp13_collection1	SReLU-heN-20	44.2%
	Mish-heN-20	43.3%
	GELU-gloU-40	21%
dblp13_venues	LReLU-heN-80	4.2%
	SReLU-heN-40	4%
	Mish-heN-20	0.4%
dblp13_people	FTS-heN-150	3.3%
	LReLU-heN-80	2.5%
	SReLU-heN-60 & ELU-gloU-60	2%

Bold text indicates the best activation function for each experiment dataset

outperformed all other activations in two out of four dataset-experiments, is a crucial novel activation. ReLU is still a competitive activation, but we have discovered others that frequently outperform it. SineReLU, FTS, Leaky-ReLU and Mish all fall into this group.

Finally, we discovered that, with the exception of ReLU and FTS, all other activation functions achieve their peak performance in a smaller number of training epochs, specifically in the range [20,80]; the precise value relies on the properties of the dataset and the model. FTS will presumably require fewer epochs if $T < 0$ is used.

In Stage D, we decided to add the HP-tuned CATA++ version that uses SineReLU activation, HeNormal weight initializer and 60 epochs for training, which was found to produce the most stable performance as it was always in the top-3 best results (Recall and NDCG metrics).

Lots of details and experiments results on HP-tuning of CATA++ before incorporating it in the present system (Stage D) can be found in our previous work [1].

4 Experiments

In this section, we present the experiments of testing and evaluating our system against the available datasets in Table 6.

To the best of our knowledge, there is not any RS of this magnitude; that is exploiting a full dataset with millions of papers, excluding the distributed or parallel ones. Regarding these systems, we could not find an open source distributed or parallel RS, or one that could be reproduced in a logical amount of time and effort (even then we would not be sure that our implementation would be the same as the original paper). Consequently, we created the three versions of our system (see Table 5) in order to run several experiments to discover the best set up (version) that presents a good performance in most cases.

4.1 Artificial users

To begin with our experiments, we created ten artificial users that would provide the appropriate input for our RS: (i) text (abstract/paragraph/keywords) of their research interests or/and a list of papers they have liked/rated/purchased previously and (ii) the publication period (papers published since a selected year). In order to generate artificial users, we conducted an extensive research on Google Scholar, Semantic Scholar, arXiv, etc., regarding the most prominent scientific areas for researchers today. So, we created a paragraph, some keywords, or/and a list of liked papers for each one of them, representing their research interests. The proposed system needs either some text or/and a list of papers that the user liked in the past. In both cases, the system will extract a list of keywords (either from the provided text or from the provided papers). This input step emulates the procedure of a user that browses in the RS's papers and provides direct feedback through clicking, reading, buying or rating papers that would provide the necessary information for the collaborative filtering and the content-based filtering.

In addition, the artificial users we created are diverse, regarding the scientific area they study. The users' input texts are listed in Table 9; most users have provided some words or phrases while User#2 and User#6 have provided as input the abstract of a paper they like. The size (length) of the provided text has no limit (depends on the users' needs and interests) for our system, due to the keywords extraction feature in the first stages, which disregards all unnecessary words (i.e., stop-words or words that have no value for the scientific area of computer science). Details are found in Sect. 3.3. Also, users with userID in the range [3,6] (Table 9) provide a list of papers they have already liked, so that the system can use this information in the CF method.

The artificial users, at the same time, provide the ground truth data utilized in calculating specific metrics (described in paragraph 2.4) for the evaluation of the system. In case the user provides some text of interest (e.g., abstract/ keywords/ full paper, etc.), the system retrieves from AMiner citation-network dataset the papers that are found to have the highest cosine similarity with the text of interest; this set of papers along with their referenced papers create a bigger set of papers that are the ground truth data for the artificial users. If the user provides a list of liked papers, then these papers along with their referenced papers create a bigger set of papers that are the ground truth data for the artificial users.

4.2 Results

We run the three RS versions for each user and compare the resulting Recall scores while making 50 recommendations each time ($k=50$). The results (Recall and NDCG scores) are presented in Tables 10 and 11, respectively.

The system version #3 achieves the best performance in 50% of the tests (5/10 users). Furthermore, the system version #3 is found to have higher median Recall value (0.4152) than

Table 9 Users' texts of research interest

UserID	Research interest
1	Bioinformatics, genes, micro-rna, dna big-data, computation and analysis
2	The goal of the interdisciplinary field of quantum computing is to comprehend how information is processed in accordance with the laws of quantum physics The use of quantum features like superposition and entanglement in computation is known as quantum computing. It has proven to be superior to traditional computing Existing computers can't compete with quantum computing's performance. It thus symbolizes a paradigm change in the history of computing technology
3	Artificial intelligence, machine learning, natural language processing
4	Recommender systems for scientific publications
5	hardware, assembly language, computer central processing unit embedded systems
6	Artificial intelligence (AI) in medical diagnosis: artificial intelligence can help healthcare professionals with a range of patient care and smart health systems. For the diagnosis of diseases, the discovery of new drugs and the identification of patient risk factors, artificial intelligence techniques that vary from machine learning to deep learning are widely used in healthcare
7	Information security, cyber-attacks, cyber-defense, cyber-security secure data storage
8	Mobile platforms, mobile communications, wireless networks internet of things
9	Robotics, machine vision, computer vision, motion sensors autonomous robots, grade of autonomy, human in the loop
10	Mathematics, algebra, geometry, tableaux, Gaussian distribution Schrödinger equations, eigenvalues and eigenvectors

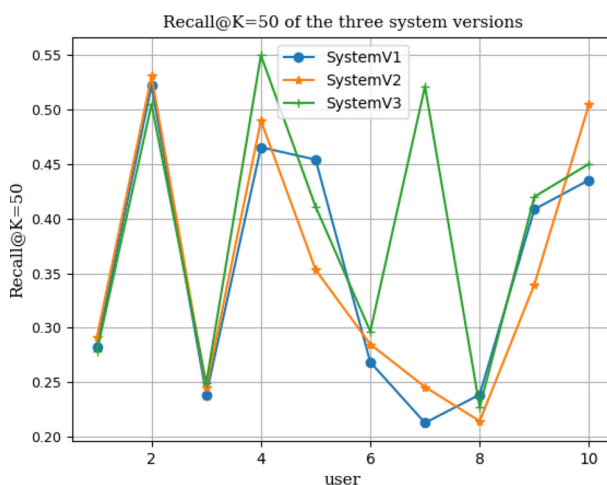
**Fig. 10** Recall score of the 3 RS versions

Table 10 Recall@K=50 of the three versions of our RS

User id	v1	v2	v3
1	0.2828	0.2913	0.2784
2	0.5215	0.5303	0.5045
3	0.2388	0.2449	0.2493
4	0.4651	0.4894	0.5494
5	0.4538	0.3531	0.4106
6	0.2685	0.2849	0.2968
7	0.2129	0.2458	0.5209
8	0.2388	0.2145	0.2274
9	0.4083	0.3397	0.4198
10	0.4351	0.5045	0.4499
Median	0.3456	0.3155	0.4152
Average	0.3526	0.3498	0.3862

Bold text indicates the best version for each user, and for median and average results

Table 11 NDCG@K=50 of the three versions of our RS

User id	v1	v2	v3
1	0.1289	0.1241	0.1294
2	0.2208	0.2356	0.2155
3	0.1363	0.1392	0.123
4	0.2017	0.2145	0.2477
5	0.2192	0.1763	0.1931
6	0.1254	0.1263	0.1348
7	0.1058	0.1115	0.2372
8	0.105	0.1021	0.1205
9	0.1831	0.1685	0.1744
10	0.1836	0.2308	0.1971
Median	0.1597	0.1539	0.1838
Average	0.1610	0.1629	0.1773

Bold text indicates the best version for each user, and for median and average results

the second better result by approximately 7%. Also, the average Recall achieved by version #3 as described in Table 10 is about 3.2% higher from the second better performance. The overall higher Recall performance of RS version #3 in most user cases (5/10) is shown in Fig. 10.

Moreover, system version #3 achieves higher NDCG scores, meaning it is able to make recommendations with a better ranking quality. Similarly, it scores the best NDCG value in 50% of the tests (5/10 users), while having the best median and average values as presented in Table 11. The overall higher NDCG performance of RS version #3 in most user cases (5/10) is shown in Fig. 11.

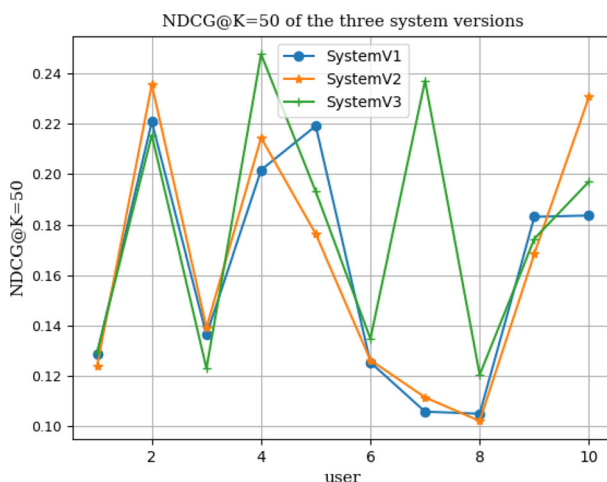


Fig. 11 NDCG score of the 3 RS versions

Our system's version #3 has an overall better Recall performance as it starts by assigning more fos to the user (i.e., 2000 fos) and this provides the system with the capability to make more diverse recommendations, covering a wider spectrum of relevant papers to the user's likes. Therefore, the system ver. #3 is not limited to recommending papers in the restricted area of the exact user's likes, but it can discover more relevant, scientific areas compared to the other two system versions. This capability is also improved as the system ver. #3 is the only version that retrieves the user's liked fos (from the papers that the user liked previously) and uses them and the fos-graph to find the strong fos connections; the system uses later those fos connections to extract papers. Finally, the NDCG scores are higher in ver. #3 as it has enabled the "highest citation bias" feature, which ranks higher on the recommendation list for the papers that have already received the most citation. This feature combined with the two features mentioned before (2000 fos and exploiting the user's liked fos) gives the system ver. #3 the capability to find more relevant papers and rank them in a more rational way; so, the system ver. #3 achieves higher NDCG scores as well.

In Figs. 10 and 11, we can see that Recall and NDCG performance, respectively, of all the system's versions changes significantly from one user to another, creating performance peaks for some experiments; for example, the system ver. #3 achieves a Recall score of 0.55 for user-experiment number 4, while it achieves only 0.25 for user-experiment number 3. This difference in performance is observed because of the size of the cluster, which the user is assigned initially. As can be seen in 4, the size of the fos-clusters created in Stage A can vary notably, because of the different amount of literature published in each scientific area. This fact drives the size of the paper collections accumulated in the next stages. The increase in the paper collections size causes a decrease in the achieved Recall and NDCG values and vice versa.

In the final stage 3.4 of our system, we incorporated CATA++ which has significantly higher performance as opposed to other models described in the literature; as a matter of fact, we incorporated the HP-tuned version of CATA++ [1], which delivered improved recommendation scores even compared to the originally published model. Overall, our system makes recommendations with an average Recall score of 0.3862 while working on a 5.3M papers

Table 12 Time complexity of the proposed RS

Function	Time
Pre-processing of AMiner citation-network	48 h
Train DL model, prediction and evaluation (10 users)	6 h
Prediction after loading trained DL model (10 users)	1 h
Mean time-to-respond per user (after loading trained DL model)	5–7 min

dataset. Other state-of-the-art RS, while working on significantly smaller datasets (CiteULike datasets in Table 3), produce significantly lower scores. Unfortunately, CATA++ and other models of the state-of-the-art literature cannot process the full AMiner Citation-Network dataset, as they would need a huge amount of memory/CPU/GPU/cluster of machines (hardware limitations).

The time complexity of the proposed RS (Table 12) is not considered high, taking into account the size of the dataset and the capabilities/characteristics of the machine (regular modern PC) where all the experiments were run.

4.3 Comparison

Our model (version 3) performance is compared to the performance of a well-known method, item-to-item collaborative filtering, as it was implemented by Linden et al. [38]; a system that, as they state, can scale to massive datasets. So, we run their algorithm on the AMiner Citation Network dataset, the same as we run the experiments of the proposed RS.

Firstly, we selected the most common authors in the dataset (top one million authors that have liked/referenced the most papers), because the dataset is extremely sparse, with the 3.5 million authors. We mention, indicatively, the original's datasets mean values: each paper has only 20 ratings and each author has about 30 papers rated that gives a sparsity of 99.999%. The process of the 1 M most common authors filtering resulted to a small reduction in the total papers, from 5.3M to 4.9M and, of course, a reduction to 1 M authors. At this stage, we have a sparsity of 99.98% which is a minor improvement. Next, we set KNN = 15, in order to keep the time needed to complete the execution of the algorithm in a reasonable scale. With this, setting the item-to-item KNN [38] needed 44 h to prepare the paper ratings vectors, each one of 1 M length, equal to the authors' number.

In the next step, we calculated the cosine similarity between the 4.9M papers in the reduced dataset and the papers of the 10 artificial users. The cosine similarity calculations and the recommendation process took about 45 min per user, depending on how many papers each user had liked; our artificial users have liked (read and rated) only 3–7 papers, which is a very low number in order to help the Item KNN algorithm in a dataset of 4.9 M publications.

Finally, we calculated Recall and NDCG while the system was making $K = 50$ recommendations (the same settings and metrics as in the proposed system's evaluation). The results in Table 13 show that the Item KNN provides really low Recall scores, because the dataset is extremely sparse and the users are quite 'new' if we consider that they have liked only a few papers in a vast dataset, so the cold start problem arises, for the user standpoint. For some of the artificial users, the Item KNN could not retrieve any of the ground truth papers, because these users have read and rated only three papers. Also, the Item KNN cannot take advantage of all the other available information in the dataset, like citation relationships, author–author relationships or textual data (title, abstract, field of study).

Table 13 Recall@K=50

User id	Item-to-item CF [38]	Proposed system v3
1	0.0	0.2784
2	0.0771	0.5045
3	0.1282	0.2493
4	0.1674	0.5494
5	0.0	0.4106
6	0.1359	0.2968
7	0.0	0.5209
8	0.0	0.2274
9	0.0945	0.4198
10	0.1396	0.4499

Item-to-item CF vs our proposed system

Table 14 NDCG@K=50

User id	Item-to-item CF [38]	Proposed system v3
1	0.0	0.1294
2	0.0948	0.2155
3	0.1135	0.123
4	0.0973	0.2477
5	0.0	0.1931
6	0.1127	0.1348
7	0.0	0.2372
8	0.0	0.1205
9	0.0897	0.1744
10	0.1092	0.1971

Item-to-item CF vs our proposed system

Nevertheless, the Item KNN algorithm provided better results in the NDCG metric; in Table 14 we can see that the scores achieved (by the Item KNN algorithm) for some users are very similar to our proposed system's scores. These results prove that its capability to place the most suitable suggestions in the highest positions inside the list of the k recommended papers is comparable in some cases to our proposed system's same capability. Specifically, as we can see in Table 14 the two systems' NDCG scores for users #3 and #6 are almost the same, which is noteworthy.

To conclude with, the comparison's results show that the proposed system outperformed the Item KNN [38] method, in both Recall and NDCG scores; the Item KNN struggled against AMiner Citation dataset due to the extreme sparsity of the dataset, the user-cold-start problem and the hidden relationships among papers or authors. We believe that the Item KNN algorithm could achieve far better results, if the artificial users had more liked/rated papers and if the data were not complex, textual data (scientific publications) with hidden relationships, but simpler data like e-commerce products.

5 Discussion of the experimental results

One of the key discoveries made throughout the experiments is the fact that the proposed RS is capable of working with a really large dataset with millions of papers and users, while running on a single, regular PC.

Also, the recorded results highlight that the performance of our RS could be further enhanced by carefully tuning some parameter values, such as changing the threshold of similarity or the use of the fos-connections network. As a result, RS version #3 outperformed the other versions of our system. It achieved a high score of Recall and NDCG in most cases, regardless of the large size of dataset and the fact that the users were intentionally different and diverse from one another. So, RS version #3 achieved Recall scores of [0.2274, 0.5494], with a median value of 0.4152 that are considered quite high values, taking into account the size of the used dataset.

Finally, we proved that our system through the fos-graph-based feature overcomes the cold start problem, because even the most recently published papers are classified in one or more fos and this way they become part of our graph (so they can be retrieved and recommended). Moreover, the use of the fos-graph helps to overcome the sparsity problem, when little or no user behavioral data are available for a user or paper; it is built irrelevantly to the number of times a paper has been read or liked.

6 Conclusion and future work

In our work, we proposed a novel, multi-staged (clustering, graph-modeling, DL) and also hybrid (CBF and CF) RS, which manages to exploit and make recommendations of the full DBLP AMiner Citation-network dataset (5.3M papers). To achieve this, we initially performed clustering of this dataset, on a field of study (fos) similarity basis. Next, we created a weighted, non-directed graph of the fos described on the above-mentioned dataset. At the final stage of our system, we embodied a DL RS (CATA++), after we tuned its Hyper-Parameters and improved its training time and performance.

The papers' volume that exists in the datasets used in the evaluation of state-of-the-art RS papers is rather downsized compared to real-world datasets available today. So, we see a huge gap between the state-of-the-art RS and real, online, digital datasets (millions or tens of millions of users and publications scale).

Our findings suggest that our RS, by incorporating many different techniques and algorithms, can deal with real-world problems and face the vast amount of available data in digital academic libraries. Especially clustering and graph-based approaches can upgrade the performance of our system's ANN/DL part. Finally, we showed that our multi-staged Academic RS can provide these results for a Million-scale dataset while running on a single, typical modern PC with medium characteristics.

Regarding future work, we plan to further improve our system by turning it into a parallel and/or distributed system, running on a number of machines or GPUs. In addition, the TF-IDF method could be replaced by more sophisticated methods to generate low-dimensional text embeddings (e.g., BERT, FastText, Word2Vec, Glove, Autoencoders, etc.) that were recently introduced by research in Natural Language Processing [39]. Moreover, it would be interesting to replace CATA++ with another DL RS providing advanced recommendation performance and accuracy. Furthermore, another step that could be modified is the clustering algorithm. Conducting a clustering analysis and evaluation of different algorithms (e.g. [40–

42]) could help us to improve the clusters produced by embodying a more efficient algorithm. Eventually, we will try to incorporate even larger academic datasets (tens or hundreds of millions of papers), whereas we may add a web crawling–scraping feature to discover and insert the most recently published papers in the system’s database.

Acknowledgements The work of M. Vassilakopoulos and A. Corral was funded by the Spanish Ministry of Science and Innovation under the R&D project HERMES (ref. PID2021-124124OB-I00).

Author Contributions Conceptualization was performed by Vaïos Stergiopoulos, Michael Vassilakopoulos, Eleni Tousidou, Antonio Corral; methodology by Vaïos Stergiopoulos, Michael Vassilakopoulos, Eleni Tousidou, Antonio Corral; investigation, software, data curation and writing—original draft preparation—by Vaïos Stergiopoulos; writing—review and editing—by Michael Vassilakopoulos, Eleni Tousidou, Antonio Corral; funding acquisition by Antonio Corral; resources by Michael Vassilakopoulos.

Funding Funding for open access publishing: Universidad de Almería/CBUA. The work of M. Vassilakopoulos and A. Corral was funded by the Spanish Ministry of Science and Innovation under the R&D project HERMES (ref. PID2021-124124OB-I00). No funds, grants, or other support was received from Vaïos Stergiopoulos and Eleni Tousidou.

Availability of data and materials Data available at <https://www.aminer.org/citation>.

Code availability Available upon request.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Stergiopoulos V, Vassilakopoulos M, Tousidou E, Corral A (2022) Hyper-parameters tuning of artificial neural networks: An application in the field of recommender systems. In: New trends in database and information systems - ADBIS 2022 proceedings. Communications in Computer and Information Science, vol. 1652, pp. 266–276. Springer, Cham. https://doi.org/10.1007/978-3-031-15743-1_25
2. Alfahhood M, Cheng J (2020) CATA++: a collaborative dual attentive autoencoder method for recommending scientific articles. IEEE Access 8:183633–183648. <https://doi.org/10.1109/ACCESS.2020.3029722>
3. Zhang Q, Lu J, Jin Y (2021) Artificial intelligence in recommender systems. Complex Intell Syst 7(1):439–457. <https://doi.org/10.1007/s40747-020-00212-w>
4. Guo Q, Zhuang F, Qin C, Zhu H, Xie X, Xiong H, He Q (2022) A survey on knowledge graph-based recommender systems. IEEE Trans Knowl Data Eng 34(8):3549–3568. <https://doi.org/10.1109/TKDE.2020.3028705>
5. Wang S, Hu L, Wang Y, He X, Sheng QZ, Orgun MA, Cao L, Ricci F, Yu PS (2021) Graph learning based recommender systems: a review. CoRR [arXiv:2105.06339](https://arxiv.org/abs/2105.06339)
6. Kreutz CK, Schenkel R (2022) Scientific paper recommendation systems: a literature review of recent publications. Int J Digit Libr 23(4):335–369. <https://doi.org/10.1007/s00799-022-00339-w>
7. Beheshti A, Yakhchi S, Mousaeirad S, Ghafari SM, Goluguri SR, Edrisi MA (2020) Towards cognitive recommender systems. Algorithms 13(8):176. <https://doi.org/10.3390/a13080176>
8. Sakib N, Ahmad RB, Haruna K (2020) A collaborative approach toward scientific paper recommendation using citation context. IEEE Access 8:51246–51255. <https://doi.org/10.1109/ACCESS.2020.2980589>

9. Sakib N, Ahmad RB, Ahsan M, Based MA, Haruna K, Haider J, Gurusamy S (2021) A hybrid personalized scientific paper recommendation approach integrating public contextual metadata. *IEEE Access* 9:83080–83091. <https://doi.org/10.1109/ACCESS.2021.3086964>
10. Li X, She J (2017) Collaborative variational autoencoder for recommender systems. In: *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, Halifax, NS, Canada, August 13–17, 2017, pp. 305–314. ACM, New York, USA. <https://doi.org/10.1145/3097983.3098077>
11. Li X, She J (2017) Relational variational autoencoder for link prediction with multimedia data. In: *Proceedings of the on thematic workshops of ACM multimedia 2017*, Mountain View, CA, USA, October 23–27, 2017, pp. 93–100. ACM, New York, USA. <https://doi.org/10.1145/3126686.3126774>
12. Hsieh C, Yang L, Cui Y, Lin T, Belongie SJ, Estrin D (2017) Collaborative metric learning. In: *Proceedings of the 26th international conference on World Wide Web, WWW 2017*, Perth, Australia, April 3–7, 2017, pp. 193–201. ACM, New York, USA. <https://doi.org/10.1145/3038912.3052639>
13. Nikzad-Khasmakhi N, Balafar MA, Feizi-Derakhshi M, Motamed C (2020) BERTERS: multimodal representation learning for expert recommendation system with transformer. *CoRR arXiv:2007.07229*
14. Eger S, Youssef P, Gurevych I (2018) Is it time to swish? Comparing deep learning activation functions across NLP tasks. In: *Proceedings of the 2018 conference on empirical methods in natural language processing*, Brussels, Belgium, October 31–November 4, 2018, pp. 4415–4424. Association for Computational Linguistics, Stroudsburg PA, USA. <https://doi.org/10.18653/v1/d18-1472>
15. Nair V, Hinton GE (2010) Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th international conference on machine learning (ICML-10)*, June 21–24, 2010, Haifa, Israel, pp. 807–814. Omnipress, Madison, USA. <https://icml.cc/Conferences/2010/papers/432.pdf>
16. Maas AL, Hannun AY, Ng AY (2013) Rectifier nonlinearities improve neural network acoustic models. In: *Proceedings of the 30th international conference on machine learning*, Vol. 28. https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf
17. He K, Zhang X, Ren S, Sun J (2015) Delving deep into rectifiers: surpassing human-level performance on imagenet classification. In: *2015 IEEE international conference on computer vision, ICCV 2015*, Santiago, Chile, December 7–13, 2015, pp. 1026–1034. IEEE Computer Society, Piscataway, NJ USA. <https://doi.org/10.1109/ICCV.2015.123>
18. Clevert D-A, Unterthiner T, Hochreiter S (2016) Fast and accurate deep network learning by exponential linear units (ELUS). *CoRR arXiv:1511.07289*
19. Rodrigues W (2018) SineReLU—an alternative to the ReLU activation function. <https://wilder-rodrigues.medium.com/sinerelu-an-alternative-to-the-relu-activation-function-e46a6199997d>. Accessed 05 Mar 2022
20. Hendrycks D, Gimpel K (2020) Gaussian error linear units (GELUS). *CoRR arXiv:1606.08415*
21. Misra D (2020) Mish: a self regularized non-monotonic activation function. *CoRR arXiv:1908.08681*
22. Ramachandran P, Zoph B, Le QV (2017) Searching for activation functions. *CoRR arXiv:1710.05941*
23. Chieng H, Wahid N, Pauline O, Perla SRK (2018) Flatten-T Swish: a thresholded RELU-Swish-like activation function for deep learning. *Int J Adv Intell Inform* 4(2):76–86. <https://doi.org/10.26555/ijain.v4i2.249>
24. Kong X, Mao M, Wang W, Liu J, Xu B (2021) Voprec: vector representation learning of papers with text information and structural identity for recommendation. *IEEE Trans Emerg Top Comput* 9:226–237. <https://doi.org/10.1109/TETC.2018.2830698>
25. Tsolakidis A, Triperina E, Sgouropoulou C, Christidis N (2016) Research publication recommendation system based on a hybrid approach. In: *Proceedings of the 20th Pan-Hellenic conference on informatics*, Patras, Greece, November 10–12, 2016, p. 78. ACM, New York. <https://doi.org/10.1145/3003733.3003805>
26. Son J, Kim SB (2018) Academic paper recommender system using multilevel simultaneous citation networks. *Decis Support Syst* 105:24–33. <https://doi.org/10.1016/j.dss.2017.10.011>
27. Kyriakidi M, Koutrika G, Ioannidis YE (2020) Recommendations as graph explorations. In: *RecSys 2020: fourteenth ACM conference on recommender systems*, Virtual Event, Brazil, September 22–26, 2020, pp. 289–298. ACM, New York, USA. <https://doi.org/10.1145/3383313.3412269>
28. Shambour Q (2021) A deep learning based algorithm for multi-criteria recommender systems. *Knowl Based Syst* 211:106545. <https://doi.org/10.1016/j.knosys.2020.106545>
29. Shi B, Tragos EZ, Ozsoy MG, Dong R, Hurley N, Smyth B, Lawlor A (2021) DARES: an asynchronous distributed recommender system using deep reinforcement learning. *IEEE Access* 9:83340–83354. <https://doi.org/10.1109/ACCESS.2021.3087406>
30. Bobadilla J, Alonso S, Hernando A (2020) Deep learning architecture for collaborative filtering recommender systems. *Appl Sci* 10(7):2441. <https://doi.org/10.3390/app10072441>
31. Li M, Zhang S, Zhu F, Qian W, Zang L, Han J, Hu S (2020) Symmetric metric learning with adaptive margin for recommendation. In: *The thirty-fourth AAAI conference on artificial intelligence, AAAI 2020*,

- New York, NY, USA, February 7–12, 2020, pp. 4634–4641. AAAI Press, Washington, USA. <https://ojs.aaai.org/index.php/AAAI/article/view/5894>
32. Dacrema MF, Boglio S, Cremonesi P, Jannach D (2021) A troubling analysis of reproducibility and progress in recommender systems research. *ACM Trans. Inf. Syst.* 39(2):20–12049. <https://doi.org/10.1145/3434185>
 33. Silva FL, Slodkowski BK, Silva KKA, Cazella SC (2023) A systematic literature review on educational recommender systems for teaching and learning: research trends, limitations and opportunities. *Educ Inf Technol* 28(3):3289–3328. <https://doi.org/10.1007/s10639-022-11341-9>
 34. Azeroual O, Koltay T (2022) RecSys pertaining to research information with collaborative filtering methods: characteristics and challenges. *Publications* 10(2):17. <https://doi.org/10.3390/publications10020017>
 35. Meijer HJA, Truong J, Karimi R (2021) Document embedding for scientific articles: Efficacy of word embeddings vs TFIDF. *CoRR* [arXiv:2107.05151](https://arxiv.org/abs/2107.05151)
 36. Tang J, Zhang J, Yao L, Li J, Zhang L, Su Z (2008) Arnetminer: extraction and mining of academic social networks. In: *Proceedings of the 14th ACM SIGKDD international conference on knowledge discovery and data mining*, Las Vegas, Nevada, USA, August 24–27, 2008, pp. 990–998. ACM, New York, USA. <https://doi.org/10.1145/1401890.1402008>
 37. Stergiopoulos V, Tsianaka T, Tousidou E (2021) Aminer citation-data preprocessing for recommender systems on scientific publications. In: *PCI 2021: 25th Pan-Hellenic conference on informatics*, Volos, Greece, November 26–28, 2021, pp. 23–27. ACM, New York, USA. <https://doi.org/10.1145/3503823.3503828>
 38. Linden G, Smith B, York J (2003) Amazon.com recommendations: item-to-item collaborative filtering. *IEEE Internet Comput* 7(1):76–80. <https://doi.org/10.1109/MIC.2003.1167344>
 39. Akritidis L, Bozani P (2023) Low-dimensional text representations for sentiment analysis NLP tasks. *SN Comput Sci* 4(5):474. <https://doi.org/10.1007/s42979-023-01913-y>
 40. Cheng X, Yan X, Lan Y, Guo J (2014) BTM: topic modeling over short texts. *IEEE Trans Knowl Data Eng* 26(12):2928–2941. <https://doi.org/10.1109/TKDE.2014.2313872>
 41. Yin J, Wang J (2014) A dirichlet multinomial mixture model-based approach for short text clustering. In: *Macassay SA, Perlich C, Leskovec J, Wang W, Ghani R (eds.) The 20th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '14*, New York, USA - August 24–27, 2014, pp. 233–242. ACM. <https://doi.org/10.1145/2623330.2623715>
 42. Zhang D, Nan F, Wei X, Li S, Zhu H, McKeown KR, Nallapati R, Arnold AO, Xiang B (2021) Supporting clustering with contrastive learning. *CoRR* [arXiv:2103.12953](https://arxiv.org/abs/2103.12953)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Vaios Stergiopoulos obtained a Bachelor's Degree in Telecommunications and Electronics Engineering from the Hellenic Air Force Academy (Icarus School) of Athens (Greece) and a Master of Science in Information Systems from the School of Science and Technology of the Hellenic Open University (Greece). Currently, he is a PhD candidate with the field of his research to be "Machine Learning in Recommender Systems for scientific publications" in the Department of Electrical and Computer Engineering of the School of Engineering of the University of Thessaly (Greece). He has been working in the Hellenic Air Force as a RADAR and Telecommunications Engineer since 2006. His research interests include Recommender Systems, Machine Learning, Data Preprocessing, Artificial Neural Networks and Deep Learning.



puting and Big Data Management.

Michael Vassilakopoulos obtained a five-year Diploma in Computer Eng. and Informatics from the University of Patras (Greece) and a PhD in Computer Science from the Department of Electrical and Computer Eng. of the Aristotle University of Thessaloniki (Greece). He has worked in the public and private sector in Greece as an Informatics Engineer. He has been with several higher institutions as an instructor, or faculty member. He also has held several academic administrative positions. Currently, he is a Professor and Head of the Department of Electrical and Computer Engineering of the University of Thessaly (Greece). He has published several papers in peer-reviewed scientific journals and conferences, chapters in books and collections and lemmas in encyclopedias and has coedited a book on Spatial Databases. He has also coordinated/participated in several R&D projects related to Databases, GIS, WWW, Information Systems and Employment. His research interests include Algorithms, Data Structures, Databases, Data Mining, Recommender Systems, GIS, Parallel and Distributed Com-



other current trends of Data Management.

Eleni Tousidou holds a Bachelor's degree in Informatics from the Department of Informatics, Faculty of Sciences of the Aristotle University of Thessaloniki (Greece) in 1996 and a PhD Diploma in Computer Science from the same department in 2002. In her Bachelor thesis she dealt with Geographical Information Systems, while her doctoral thesis was on Databases, namely on "Signature Structures in Complex Object Retrieval". In 2001 she was a visiting researcher at the Department of Computer Science at the University of Alberta, Canada. She has participated in R&D projects related to Databases and GIS and has worked as a secondary school teacher for several years. From 2002 to 2017, she worked as an adjunct professor at the department of Electrical and Computer Engineering of the University of Thessaly. Since 2017, she works as Laboratory Teaching Staff at the same department. She has published articles in scientific journals and conferences and has reviewed conference articles. Her research interests focus on Databases, Algorithms, Text Analysis, Data Mining, GIS, Big Data and



Antonio Corral Antonio Corral is Full-Professor with the Department of Informatics at the University of Almeria (Spain). He received his PhD in Computer Science from the University of Almeria (Spain). He has participated actively in several research projects in Spain (INDALOG, vManager, ENIA, CoSmart, HERMES, etc.) and Greece (CHOROCHRONOS, ARCHIMEDES, etc.). He has published (>100 articles) in referred scientific international journals (Data & Knowledge Engineering, GeoInformatica, Information Sciences, Computer Standards & Interfaces, JSS, KBS, FGCS, Computing, Cluster Computing, DAPD, IoT, IJPP, SPE, etc.), peer-reviewed conferences (SIGMOD, SSD, SAC, ADBIS, SOFSEM, DEXA, OTM, MEDI, MEDES, etc.) and several scientific book chapters. His main research interests include databases, data structures, algorithms, query processing, indexing, parallel and distributed computing and other current topics related to data management.